Pico 2 Flight Software Testing Guide

This guide details how to compile and run specific verification test loops directly on the Raspberry Pi Pico 2.

We use **Cargo Feature Flags** to isolate these tests. This guarantees that test code is completely isolated and never accidentally compiled into the production flight software binary. Once you run a given test command, the microcontroller will bypass its normal flight loop and execute the isolated test loop indefinitely until disconnected.

🚀 Getting Started (For New Users)

If you have entirely fresh hardware and want to run these tests, follow these steps:

1. **Clone the Repository**

   ```
   git clone https://github.com/cornellrocketryteam/Software25-26.git
   cd Software25-26/fsw
   ```

2. **Checkout the FSW Branch**

   ```
   git checkout fsw-spring26
   ```

3. **Install Dependencies** Ensure you have Rust and the `picotool` toolchain installed for your Pico 2. On a Mac, you can install this using Homebrew:

   ```
   brew install picotool
   ```

4. **Connect the Hardware** Plug the Raspberry Pi Pico 2 into your computer via USB. Hold the `BOOTSEL` button on the board while plugging it in.

🛠️ Hardware Testing Commands

Run the following commands while inside the `fsw/` directory.

1. Combined Full Hardware Sequence

Tests all major sensors, actuates the Main Valve, actuates the Solenoid Valve, and fires the Drogue parachute in a repeating loop.

```
cargo run --features "test_hw_all"
```

2. Main Actuation Valve (MAV)

Repeatedly actuates the MAV Open for 2.5 seconds, then actuates it Closed, and waits 5 seconds before repeating.

```
cargo run --features "test_mav"
```

3. Solenoid Valve (SV)

Repeatedly actuates the SV Open for 2 seconds, then actuates it Closed, and waits 5 seconds before repeating.

```
cargo run --features "test_sv"
```

4. Parachutes / Solid State Arrays (SSA)

Fires the Drogue Chute pin for 2 seconds, then immediately fires the Main Chute pin for 2 seconds, waiting 5 seconds before repeating.

```
cargo run --features "test_ssa"
```

5. Buzzer

Triggers the buzzer 3 times, waits 2 seconds, then triggers the buzzer 2 times, waiting 5 seconds before repeating.

```
cargo run --features "test_buzzer"
```

6. Sensor Verification (Telemetry Spam)

The fastest way to verify all sensor connections. Bypasses the flight state machine and spams a rapid stream of direct readings from the BMP390, GPS, IMU, Magnetometer, and ADC directly to your computer console.

```
cargo run --features "test_sensors"
```

💻 Simulation Testing Commands

Software-in-the-loop tests validate the logic transitions without requiring physical sensors.

1. Simple Flight Simulation

Simulates an ideal flight profile with normal state transitions.

```
cargo run --features "sim_simple"
```

2. Fault Simulation

Injects sensor faults to ensure the flight state transitions to Fault correctly.

```
cargo run --features "sim_fault"
```

3. Stability Simulation

Validates that the software doesn’t unexpectedly transition to a mode when the condition isn’t satisfied.

```
cargo run --features "sim_stability"
```

4. Extra Features Simulation

Tests MAV timeout, manual mode changes, and payload communication works with flight loop.

```
cargo run --features "sim_extra"
```

5. Hardware-in-the-loop Simulation (HSIM)

Fakes the altimeter data while running the flight loop and actuating the hardware.

```
cargo run --features "sim_hsim"
```

6. QSPI Flash Storage Simulation

Writes mock flight packet data to the onboard Flash memory chip and reads it back to verify data.

```
cargo run --features "sim_flash"
```

7. Combined Full Simulation Sequence

Runs all of the above simulations sequentially.

```
cargo run --features "sim_all"
```

🌐 Unified Testing Command

Test Everything (Software & Hardware)

Sequentially runs all software simulations (`sim_all`) and then enters the continuous hardware testing loop (`test_hw_all`).

```
cargo run --features "test_all"
```

✈️ Returning to Normal Flight Mode

To compile and load the actual, production Flight Software state machine, simply run `cargo run` without supplying any test feature flags:

```
cargo run
```